Geavanceerde Computerarchitectuur

Lab Report – Session 1: GPU Programming Basics

1 Context

The goal of this lab session is to get acquainted with GPU programming using CUDA. The central task is to implement a *parallel array reversal* on the GPU: given an input array of integers, produce an output array in which the elements appear in reverse order.

Beyond the basic reversal, we explored several variants of the implementation to understand how different grid/block configurations affect correctness and performance, and we experimented with a grid-stride loop pattern to decouple the problem size from the number of launched threads.

2 Methodology

2.1 CUDA Workflow

Every CUDA program follows the same general structure: (1) allocate memory on the device with `cudaMalloc`, (2) copy data from host to device with `cudaMemcpy`, (3) launch the kernel, (4) copy results back from device to host, and (5) free device memory.

2.2 Thread Indexing

A key concept introduced in the session is how a thread determines which array element it is responsible for. Because `threadIdx.x` only provides a *relative* position within a block, we must compute a global (absolute) index:

$$\text{idx} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x} \tag{1}$$

This formula is used in all reversal kernels. Without the block offset, multiple blocks would overwrite the same elements while leaving others untouched.

2.3 Version 1 – Single Block (`reverse.cu`)

The first implementation launches a single block of N threads (`reverse<<<1, N>>>`). Each thread i writes `a[N-1-i]` into `b[i]`, effectively reversing the array. A bounds check (`if (idx <= n)`) guards against out-of-bounds accesses. This version is straightforward but does not make use of multiple blocks, which is required by the lab goal.

2.4 Version 2 – Multiple Blocks (`reversemulti.cu`)

This version introduces multiple blocks by parametrising the launch configuration with M blocks and $N/M \cdot M$ threads per block. Basic CUDA event timing was added here to measure kernel execution time. The same kernel body is reused, demonstrating that the indexing formula generalises across multi-block configurations without modification.

2.5 Version 3 – Benchmarking Fixed Grid (`speedtest.cu`)

To answer the question of how grid and block sizes affect performance, this version systematically benchmarks a range of configurations (blocks $\in \{1, 2, 4, 8, 16\}$, threads per block $\in \{1, 2, 4, 8, 16, 32, 64\}$) for a fixed array of $N = 100$ elements. Configurations where the total thread count is less than N are skipped, since in this kernel each thread handles exactly one element and any uncovered element would be left unwritten. The benchmark uses `cudaEvent_t` for precise GPU-side timing.

2.6 Version 4 – Grid-Stride Loop (`gridstride.cu`)

The most flexible version replaces the one-thread-one-element model with a *grid-stride loop*:

```
for (int i = idx; i < N; i += blockDim.x * gridDim.x) {
    b[i] = a[N - 1 - i];
}
```

Here the stride equals the total number of threads in the grid (`blockDim.x * gridDim.x`). This means the kernel remains correct regardless of whether the grid is smaller or larger than the problem size: if the grid is smaller, each thread simply processes multiple elements; if it is larger, the extra threads exit the loop immediately. This version also queries and prints hardware properties (`cudaGetDeviceProperties`) such as the maximum threads per block and the number of streaming multiprocessors (SMs), providing physical context for the CUDA grid abstraction.

3 Observations and Results

3.1 Correctness of the Reversal

All versions produce the expected output: for an input `[0, 1, 2, ..., N-1]`, the output is `[N-1, N-2, ..., 1, 0]`. The bounds check `idx < N` (or `idx <= n` in the early versions – note that `<=` is slightly incorrect and would allow one extra thread to write to `b[n]`, which is out-of-bounds; the correct guard is `<`) is essential when the total number of launched threads exceeds N.

3.2 Open Question 1 – Choosing Block and Grid Sizes

The session asked how to divide work across blocks and threads. From our experiments, either a single block of N threads or multiple blocks each with N/M threads both work correctly, provided the indexing formula accounts for the block offset. A practical consideration is that the maximum number of threads per block on most modern Nvidia GPUs is 1024. For problem sizes larger than 1024, multiple blocks become mandatory rather than optional.

3.3 Open Question 2 – Correct Indexing

Without the block offset term, each block would independently index from 0 to `blockDim.x - 1`, causing all blocks to process the same first chunk of the array and leaving subsequent chunks untouched. The global index formula resolves this by shifting each block's indices by the appropriate offset.

3.4 Grid-Stride vs. Fixed Grid

The grid-stride loop in `gridstride.cu` is a more robust and reusable pattern than the one-thread-one-element model. It decouples the kernel launch configuration from the problem size,

which is useful when the problem size is not known at compile time or when the optimal number of threads is determined by hardware occupancy rather than the data size.

3.5 Timing and Performance

The timing results from `speedtest.cu` and `reversemulti.cu` show that for small arrays (e.g. $N = 100$), kernel execution times are in the sub-millisecond range and differences between configurations are largely dominated by launch overhead rather than computation. This underlines an important lesson: for small workloads, CPU execution may be more efficient; the GPU's advantage emerges for large, highly parallel workloads.

4 Conclusion

This session introduced the fundamental building blocks of CUDA GPU programming. We implemented an array reversal in several iterations, progressing from a single-block kernel to a multi-block version and finally to a grid-stride loop. The key takeaways are:

- Global thread indexing (`blockIdx.x * blockDim.x + threadIdx.x`) is essential for correct multi-block kernels.
- The choice of grid and block size must respect hardware limits (e.g. max 1024 threads per block) and cover the full problem size.
- The grid-stride loop pattern produces more flexible and robust kernels.
- Memory management (allocation, host-to-device copy, device-to-host copy, freeing) is an explicit responsibility of the programmer in CUDA.
- For small workloads, GPU overhead is non-negligible; the GPU excels at large-scale parallelism.

A Source Code

A.1 reverse.cu – Single-Block Reversal

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel to add two arrays
__global__ void reverse(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (idx <= n) {
        b[idx] = a[n-idx-1];
    }
}



int main() {
    const int N = 20;
    int a[N], b[N];
    int *d_a, *d_b;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel (1 block of N threads)
    reverse<<<1, N>>>(d_a, d_b, N);

    // Copy result back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);

    // Print results
    std::cout << "Result: ";
    for (int i = 0; i < N; i++) {
        std::cout << b[i] << " ";
    }
    std::cout << std::endl;

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

A.2 reversemulti.cu – Multi-Block Reversal with Timing

```cpp
#include <iostream>
```

```cpp
#include <cuda_runtime.h>
#include <math.h>


// CUDA kernel to add two arrays
__global__ void reverse(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    if (idx <= n) {
        b[idx] = a[n-idx-1];
    }
}



int main() {
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);
    cudaEventRecord(start_cuda);

    int N = 8;
    const int M = 2;
    int devide = N/M * M;
    int a[N], b[N];
    int *d_a, *d_b;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel (1 block of N threads)
    reverse<<<M, devide>>>(d_a, d_b, N);

    // Copy result back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);

    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);
    float ms;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
    std::cout << ms << "␣";

    std::cout << std::endl;

    // Print results
    std::cout << "In:␣";
    for (int i = 0; i < N; i++) {
        std::cout << a[i] << "␣";
```

```
    }
    std::cout << std::endl;

    // Print results
    std::cout << ": ";
    for (int i = 0; i < N; i++) {
        std::cout << b[i] << " ";
    }
    std::cout << std::endl;

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

A.3 speedtest.cu – Benchmarking Fixed-Grid Kernel

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>

__global__ void reverse(int* a, int* b, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) { //Extra threads fail here and exit
        b[idx] = a[N - 1 - idx];
    }
}

int main() {
    int N = 100 ;
    std::vector<int> a(N), b(N);

    for (int i = 0; i < N; i++) a[i] = i;

    int *d_a, *d_b;
    cudaMalloc(&d_a, N * sizeof(int));
    cudaMalloc(&d_b, N * sizeof(int));

    cudaMemcpy(d_a, a.data(), N * sizeof(int), cudaMemcpyHostToDevice);

    // Test different configurations
    std::vector<int> blocks_to_test   = {1, 2, 4, 8, 16};
    std::vector<int> threads_to_test  = {1, 2, 4, 8, 16, 32, 64};

    std::cout << "Benchmarking reverse kernel\n";
    std::cout << "N = " << N << "\n\n";

    for (int M : blocks_to_test) {
        for (int divide : threads_to_test) {

            if (M * divide < N) continue;  // skip invalid configs, our
                grid does not cover the entire problem space

            cudaEvent_t start_cuda, stop_cuda;
            cudaEventCreate(&start_cuda);
            cudaEventCreate(&stop_cuda);
```

```
            cudaEventRecord(start_cuda);

            reverse<<<M, divide>>>(d_a, d_b, N);

            cudaDeviceSynchronize();

            cudaEventRecord(stop_cuda);
            cudaEventSynchronize(stop_cuda);

            float ms = 0;
            cudaEventElapsedTime(&ms, start_cuda, stop_cuda);

            cudaMemcpy(b.data(), d_b, N * sizeof(int),
                cudaMemcpyDeviceToHost);

            std::cout << "Blocks (M): " << M
                      << " | Threads/block: " << divide
                      << " | Time: " << ms << " ms\n";

            cudaEventDestroy(start_cuda);
            cudaEventDestroy(stop_cuda);
        }
    }

    std::cout << "\nFinal output check:\n";
    std::cout << "Input:  ";
    for (int i = 0; i < N; i++) std::cout << a[i] << " ";
    std::cout << "\nOutput: ";
    for (int i = 0; i < N; i++) std::cout << b[i] << " ";
    std::cout << "\n";

    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

A.4 gridstride.cu – Grid-Stride Loop Kernel

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>

__global__ void reverse(int* a, int* b, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Grid-stride loop
    for (int i = idx; i < N; i += blockDim.x * gridDim.x) {
        b[i] = a[N - 1 - i];
    }
}


int main() {
    cudaDeviceProp prop; int device = 0; cudaGetDeviceProperties(&prop,
       device); std::cout << "GPU Name: " << prop.name << "\n"; std::
       cout << "Max threads per block: " << prop.maxThreadsPerBlock << "
```

```
    \n"; std::cout << "Max␣block␣dimensions:␣" << prop.maxThreadsDim
    [0] << "␣x␣" << prop.maxThreadsDim[1] << "␣x␣" << prop.
    maxThreadsDim[2] << "\n"; std::cout << "Max␣grid␣dimensions:␣" <<
     prop.maxGridSize[0] << "␣x␣" << prop.maxGridSize[1] << "␣x␣" <<
    prop.maxGridSize[2] << "\n"; std::cout << "Multiprocessors␣(SMs):
    ␣" << prop.multiProcessorCount << "\n";
int N = 100 ;
std::vector<int> a(N), b(N);

for (int i = 0; i < N; i++) a[i] = i;

int *d_a, *d_b;
cudaMalloc(&d_a, N * sizeof(int));
cudaMalloc(&d_b, N * sizeof(int));

cudaMemcpy(d_a, a.data(), N * sizeof(int), cudaMemcpyHostToDevice);

// Test different configurations
std::vector<int> blocks_to_test   = {1, 1, 2};
std::vector<int> threads_to_test  = {1, 2, 4};

std::cout << "Benchmarking␣reverse␣kernel\n";
std::cout << "N␣=␣" << N << "\n\n";

for (int M : blocks_to_test) {
    for (int divide : threads_to_test) {

        //if (M * divide < N) continue;  // skip invalid configs,
            our grid does not cover the entire problem space

        cudaEvent_t start_cuda, stop_cuda;
        cudaEventCreate(&start_cuda);
        cudaEventCreate(&stop_cuda);

        cudaEventRecord(start_cuda);

        reverse<<<M, divide>>>(d_a, d_b, N);

        cudaDeviceSynchronize();

        cudaEventRecord(stop_cuda);
        cudaEventSynchronize(stop_cuda);

        float ms = 0;
        cudaEventElapsedTime(&ms, start_cuda, stop_cuda);

        cudaMemcpy(b.data(), d_b, N * sizeof(int),
            cudaMemcpyDeviceToHost);

        std::cout << "Blocks␣(M):␣" << M
                  << "␣|␣Threads/block:␣" << divide
                  << "␣|␣Time:␣" << ms << "␣ms\n";

        cudaEventDestroy(start_cuda);
        cudaEventDestroy(stop_cuda);
    }
}

```

```
    std::cout << "\nFinal output check:\n";
    std::cout << "Input: ";
    for (int i = 0; i < N; i++) std::cout << a[i] << " ";
    std::cout << "\nOutput: ";
    for (int i = 0; i < N; i++) std::cout << b[i] << " ";
    std::cout << "\n";

    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

References

[1] Anthropic, *Claude Sonnet 4.6* [Large language model], 2025. Available at: `https://claude.ai`